

Mind the Middle Layer: A Design Framework and Taxonomy of Mental-Model Breakdowns in LLM-Agent-Controlled Robots

Anonymous Author(s)

Abstract

Natural-language control lets non-experts command physical robots through a large language model (LLM) agent that interprets the request, synthesizes a control program, validates it, and deploys it to the robot’s microcontroller. The synthesis-and-deployment layer is perceptually invisible: the user speaks, and the robot moves. We argue that this invisibility produces a specific, predictable failure of understanding, which we call *layer collapse*: because users cannot perceive the middle layer, they compress a multi-stage causal chain into a single intending agent—the robot—and every subsequent judgement about capability, fault, and reliability inherits that compression. This paper contributes (i) the layer-collapse account and a taxonomy of seven mental-model breakdowns organized along two dimensions, the pipeline layer where the true cause lives and the class of belief the user gets wrong; (ii) a system design that makes the taxonomy addressable, including a capability handshake, a two-tier execution model in which the default path constrains generation to an allow-listed primitive API running on a pre-audited firmware runtime, and a validate-before-deploy stage; (iii) a preregisterable, two-phase study protocol with a specified mental-model-accuracy instrument, designed to test the taxonomy rather than to confirm it; and (iv) a factorial evaluation plan for six mitigations tied to human–AI interaction guidelines. This is a theoretical and artifact contribution: no participant data have been collected, and we mark explicitly what the framework predicts versus what remains untested.

CCS Concepts

• **Human-centered computing** → **Human computer interaction (HCI)**; **HCI design and evaluation methods**; • **Computer systems organization** → *Embedded and cyber-physical systems*.

Keywords

mental models; folk theories; human–robot interaction; large language models; LLM agents; end-user robot programming; code generation; trust calibration; explainability; requirements engineering

1 Introduction

Controlling a robot no longer requires writing firmware by hand. A growing class of systems lets a person issue a spoken or typed command to an LLM agent that parses the intent, synthesizes control code for that specific robot, and runs it on the robot’s microcontroller [1, 29, 42, 45]. The observable interaction is short: the user talks, and after a pause the robot acts. The actual chain contains several hidden steps—intent interpretation, program synthesis, validation, compilation, and deployment to hardware—none of which is visible at the interface.

Invisibility is the design problem. An interactive artifact succeeds when its *system image* lets a user bridge the gulf of execution and the gulf of evaluation [35]. A pipeline that hides its middle layer widens the gulf of evaluation: when the robot does something unexpected, the user has no perceptual basis for locating the cause. Mental-model theory predicts what follows. A mental model is a person’s runnable account of how a system works; it is typically incomplete and unstable, yet it governs action and expectation [20, 22, 36].

1.1 The claim: layer collapse

Prior work on invisible computation documents that people build folk theories of layers they cannot see [14, 15, 40]. We make a sharper, structural claim specific to this pipeline. Users of an agent → code → hardware system do not merely hold an incomplete model of the middle layer; they hold a model in which the middle layer *does not exist as a separate causal locus*. The chain {perception, reasoning, synthesis, validation, deployment, actuation} collapses into a single node, “the robot,” which understands and chooses (Figure 1). We call this *layer collapse*, and we argue it is generative rather than descriptive: once the intermediate layers are absent from a person’s model, a set of downstream errors follows as a consequence rather than as independent phenomena.

- A user cannot attribute a fault to a layer that is not in their model, so distinct faults—a muted microphone, an ungrounded generation, a failed flash, a stalled motor—collapse into one verdict, “it doesn’t work.”
- A user with no model of a bounded capability manifest infers capability from linguistic expressibility rather than from hardware.
- A user who locates intelligence in the effector reads variance in generator competence (across input language, across board family) as variance in the robot’s condition.
- A user modelling one agent rather than a versioned service reads a silent model update as robot malfunction.

This is a testable causal claim, not a restatement of the categories: it predicts that interventions restoring the *structure* of the pipeline should reduce attribution errors across several categories at once, whereas interventions that add explanation without restoring structure should not. Section 7 operationalizes that prediction.

1.2 Scope, research questions, contribution

The unit of analysis is an individual non-expert operating a single small robot—a wheeled or simple manipulator platform driven by an ESP32- or Arduino-class microcontroller—through an integrated agent. Primary users have little or no robotics or programming background; hobbyists and expert developers are comparison groups. Because the effector is physical, a wrong model is not merely inconvenient: misjudging what the robot will do, or mis-attributing why it failed, can produce unsafe actions or the discarding of working hardware. This distinguishes the problem from mental-model work

on screen-based recommender and conversational systems, where the stakes are largely informational [14–16].

RQ1 Which mental-model breakdowns do non-expert users form of an agent → synthesis → hardware pipeline, and do they cluster by pipeline layer and by prior technical exposure?

RQ2 Does layer collapse mediate those breakdowns—that is, does a user’s structural model of the pipeline predict their fault-attribution accuracy?

RQ3 Which mitigations repair the model and improve appropriate reliance, and does structural transparency outperform equal-dose explanation that leaves the structure implicit?

Status of this work and contribution type. This paper makes a theoretical contribution and an artifact contribution [46]; it is not an empirical report. No participant data have been collected. We state this once here rather than hedging throughout: every incidence figure, persona, or quote is absent from this paper by design, and Sections 6 and 7 are protocols to be preregistered, not results. The claims we do make are (a) conceptual—that layer collapse organizes an otherwise unstructured list of failure modes—and (b) design—that a capability handshake, a two-tier execution model, and layer-specific failure reporting make each taxonomy category addressable at the interface. Readers should judge the taxonomy on its derivation and coverage, not on incidence.

2 Related Work

2.1 Mental models and folk theories of hidden computation

Users of algorithmically curated systems routinely construct folk theories to explain behaviour whose mechanism they cannot observe [14, 40], and revealing a previously invisible layer measurably reshapes those theories [15]. Models of an AI agent are formed from observed behaviour rather than from knowledge of the underlying technology [16], which is precisely why a system whose visible behaviour is “robot moves” invites a model in which the robot is the reasoner. Mental-model soundness affects how well users can personalize and debug intelligent systems [26], and explanation dose is non-monotonic: more explanation is not uniformly better [27]. Expectation-setting interventions can adjust what users believe an imperfect AI will do [24], and mental-model quality is linked to human–AI team performance [4]. Conversational agents in particular produce a documented gulf between expectation and experience [32], and unannounced algorithmic change violates stable models and provokes resistance [11]. Recent work argues that LLM-based systems demand a rethought transparency agenda [30].

2.2 Trust, failure, and attribution in HRI

Automation should be designed for *appropriate reliance*, since both over- and under-trust are harmful [28, 37]. Trust in robots is shaped most strongly by robot performance characteristics [17], degrades measurably with observed failures, and is sensitive to the feedback offered about them [10, 41]. HRI has a developed vocabulary for failure types and their resolution [19] and for mental modelling in human–robot teams [43]. Crucially for this paper, autonomy

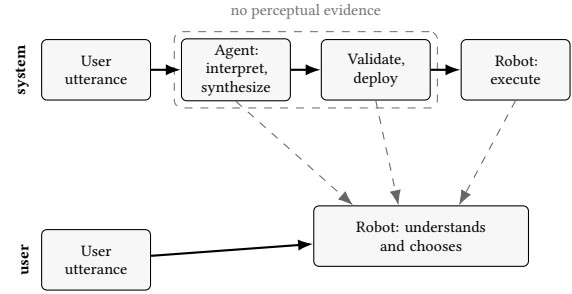


Figure 1: Layer collapse. The system is a chain of causally distinct stages (top); the middle stages produce no perceptual evidence, so the user’s model compresses them into a single intending effector (bottom). Dashed arrows mark the compression. The taxonomy in Section 5 enumerates its consequences.

and transparency change *whom users blame* when a robot errs [23], and legible motion is motion from which an observer can infer intent [12]. Our contribution is to carry this attributional lens across a layer HRI has not had to consider before: a language model that writes the robot’s program at run time.

2.3 End-user robot programming and LLM code generation

End-user robot programming has long sought to lower the authoring barrier [2]. LLM-based approaches now generate control programs directly: Code as Policies composes perception outputs and control-primitive APIs into executable programs [29]; ProgPrompt emits situated plans with precondition checks [42]; SayCan grounds instructions in a robot’s affordances, noting that ungrounded LLMs otherwise produce actions that are nonsensical or unsafe for a physical situation [1]; practitioner guidance documents combining a high-level function library with real robot APIs [45]. On the embedded side, generating, compiling, and flashing Arduino/ESP32 firmware from natural language exists in open tools [5]. Generated code is imperfect in ways users systematically under-detect [7, 38], and developers’ expectations of code-generation tools diverge from what those tools actually do [44]. Generation quality is also uneven across input languages, with markedly worse performance for lower-resource languages [21], and long-context models use their context unevenly [31].

2.4 The gap

Each capability in this pipeline exists and should be reused rather than reinvented. What is missing is a human-centered account of the composite: the end user of an integrated agent → synthesis → hardware system faces a causal chain that no prior mental-model study covers, because screen-based work has no deployment layer and prior HRI work has no run-time program synthesis. Section 5 supplies that account.

3 Design Space and Requirements

3.1 Stakeholders and propositions

Requirements discovery identifies the people, activities, values, and constraints that define the design space [39]. *Primary users* are non-expert operators who issue commands and interpret outcomes. *Secondary users* are hobbyist makers and expert developers, who serve as comparison groups and may extend the system. *Deployers*—educators, lab supervisors, product integrators—install the agent on a robot and are accountable for its behaviour. *Domain experts* in robotics and NLP ground the capability model. Their interests diverge: a primary user values predictability and clear recovery, a deployer must additionally satisfy safety, and an integrator must manage the cost and latency of a remote model.

We state three propositions, each derived from the layer-collapse account and each falsifiable by the study in Section 6. **P1:** users will attribute intent and understanding to the robot rather than to the hidden agent. **P2:** users will not, unaided, distinguish a hardware fault from a synthesis fault. **P3:** users will infer capability from what is expressible in language rather than from the robot’s hardware. P1–P3 are consistent with prior findings that people are unaware of invisible computational layers and build folk theories to explain visible behaviour [14, 15]; they are hypotheses about *this* system, and the study is designed so that each can fail.

3.2 Requirements

Table 1 lists the core requirements. Each names an actor, a condition, and a success criterion, and is grounded in literature rather than designer preference. Several are direct implications of established human–AI interaction guidelines—make clear what the system can do and how well, convey why it did what it did, support efficient correction, and notify users about change [3]—and of designing for appropriate reliance [28].

4 System Design

4.1 What this design adds

The synthesis, grounding, and flashing mechanisms are reused from the systems cited in Related Work. Three elements are ours and should be stated plainly rather than dressed as invention: (i) a plug-and-play capability handshake that turns device registration into a manifest the agent uses to refuse gracefully (R1–R2); (ii) a two-tier execution model that keeps arbitrary firmware synthesis off the default path (R3–R4); and (iii) an interface contract that every failure is reported with its layer (R6), which is what makes the taxonomy addressable rather than merely descriptive.

4.2 Modular architecture

Figure 2 shows the architecture. The **Device/Peripheral** layer performs auto-detection and a configuration handshake on connect, analogous to USB HID enumeration, producing a **capability manifest** (R1). The **Input/Perception** layer handles microphone, camera, and sensors, and reports a specific absence rather than proceeding when a required sensor is missing (R6). The **Agent/Reasoning** layer parses intent, plans, and emits an artifact; it supports a local on-device model and a remote API model, with a connectivity handler managing retries, timeouts, offline fallback, and a latency

budget. The **Validation** layer applies the four checks of R4. The **Deployment** layer either binds the artifact to the resident runtime (tier 1) or compiles and flashes firmware (tier 2), accounting for toolchain differences between the ESP-IDF/Arduino-core path and the AVR path. The **Execution & Feedback** layer confirms whether the action occurred and returns layer-tagged telemetry to both the agent and the user (R5–R6).

4.3 Two-tier execution

The original formulation of this system had the agent emit firmware directly, with static and “dry-run” checks before flashing. We revise it, because that arrangement cannot support the safety claim it makes: static analysis of generated C++ does not establish that a motor command is bounded, and a dry run without a physics model does not establish that a trajectory is safe. In **tier 1**, the default, a fixed audited runtime is flashed once at provisioning and exposes an allow-listed primitive API (bounded drive, turn, wait, read, and loops with static iteration bounds); the agent emits a program over those primitives, which is parsed, type-checked against the manifest, bounds-checked, dry-run in a kinematic simulator, and then interpreted—never flashed—with interlocks enforced in the runtime rather than in generated code. In **tier 2**, entered explicitly when new capability is required, full firmware synthesis and flashing occur behind the same validation gate plus deployer review. The tiering is also a research instrument: it separates the *deploy* layer from the *synthesis* layer in users’ experience, and Section 7 uses that separation.

4.4 Operational flow

Figure 3 traces one command. Its distinctive property is that each way an episode can fail exits through a different labelled report, so the interface can name the layer (R6). This is the design-side counterpart to WM2 (§5): the system distinguishes fault types precisely because users, unaided, cannot.

5 A Taxonomy of Mental-Model Breakdowns

5.1 Construction, and what kind of object this is

The taxonomy is derived by crossing two enumerations. The first is the *failure surface* of the architecture in §4: every edge in Figure 3 at which an episode can terminate abnormally, together with the cross-session failure of version drift. The second is the set of *documented breakdown mechanisms* in prior work: folk-theory formation over invisible layers [14, 15, 40], mis-calibrated trust [28, 37], social attribution to machines [23, 34], and known limits of LLM-driven systems [1, 21, 31]. A category is admitted when a failure-surface element and a breakdown mechanism jointly predict a specific misattribution.

Three properties should be stated rather than assumed. The categories are *not mutually exclusive*: one episode can instantiate several, and the coding scheme in §6 permits multiple labels per segment. Coverage is *relative to this architecture*, not to natural-language robot control in general; a system without a deploy step has no WM2-deploy variant. And the set is a *coding frame* intended to be revised: the study’s inductive layer exists to add, split, or retire categories, and we would treat a category that never appears as an informative result rather than a nuisance.

Table 1: Core requirements. IDs are stable and referenced from the system design (§4), the taxonomy (§5), and the evaluation (§7).

ID	Type	Requirement	Grounding	Priority
R1	Functional	On robot connect, the system shall auto-register device identity and a machine-readable capability manifest (sensors, actuators, board family) without user configuration.	Plug-and-play lowers the barrier for non-experts and grounds later refusal [2].	Must
R2	Functional	The agent shall refuse, with a layer-attributed explanation, any command outside the registered manifest, rather than attempting a partial or fabricated action.	Ungrounded generation is unsafe or nonsensical for a physical situation [1]; guideline G1 [3].	Must
R3	Safety	The default execution tier shall constrain generation to an allow-listed primitive API interpreted by a pre-audited firmware runtime; free-form firmware synthesis shall be a separate, explicitly entered tier.	Composing audited primitives rather than emitting arbitrary firmware [29]; generated code is imperfect and under-inspected [7, 38].	Must
R4	Safety	No artifact shall reach the actuators before passing (i) a manifest type-check, (ii) static bounds on actuation duration and current, (iii) a simulated dry run, and (iv) runtime interlocks (watchdog, current limit, hardware e-stop).	Static analysis alone cannot establish physical safety; interlocks must be in the audited runtime, not in generated code.	Must
R5	Data	The system shall retain per-episode provenance: utterance, parse, generated artifact, validation verdict, deploy status, execution telemetry, and model/firmware version.	Enables separating user confusion from genuine system failure (§8).	Must
R6	Quality	The interface shall report every failure with a distinguishable, layer-specific cause: input, capability, synthesis, deploy, or execution.	Users cannot otherwise localize faults (P2); supports appropriate reliance [28] and shapes attribution [23].	Must
R7	Quality	The interface shall make the pipeline structure legible on demand (“interpreting,” “writing program,” “checking,” “uploading,” “running”).	Revealing an invisible layer reshapes models [15]; guideline G11 [3].	Should
R8	Quality	The system shall expose session/context state so a user can see when earlier commands are no longer referenceable.	Context limits silently break follow-ups [31].	Should
R9	Quality	The system shall present a reliability cue calibrated to observed per-category success rather than to recent streaks.	Early success inflates trust [28]; expectation adjustment works [24].	Should
R10	Organizational	When the underlying model or firmware changes, the deployer shall be able to surface a change notice with a behavioural diff.	Silent change violates expectations and triggers resistance [11]; guideline G18 [3].	Could

5.2 Two dimensions

Each category is located by *where the true cause lives* (the pipeline layer) and *which class of belief the user gets wrong*. Four belief classes follow from layer collapse:

Structural beliefs about which components exist and where reasoning happens (WM6).

Attributional beliefs about why *this* episode failed (WM1, WM2, WM4).

Boundary beliefs about what the system can do at all (WM3).

Temporal beliefs about how consistently it works, now and across sessions (WM5, WM7).

The ordering is not arbitrary. Structural error is upstream: a user who does not represent the middle layer has no vocabulary in which an attributional judgement could be correct. This is the empirical prediction of RQ2, and it separates our account from a flat list of failure modes.

5.3 The categories

WM6 (structural) — “*the robot understood me and chose.*” *Cause:* reasoning happens in the agent; the robot executes. *Scenario:* the user addresses the robot, adapts phrasing to it, and reads its motion as deliberate. *Why misattributed:* people respond socially to

machines and attribute understanding and intention [34]; models are built from observed behaviour, not from the underlying technology [16]; and autonomy and transparency measurably shift attribution of blame [23]. We list this first because it is the collapse itself; the remaining categories are its consequences.

WM1 (attributional, agent layer) — “*it’s broken / it got dumber.*” *Cause:* generation competence is uneven across input language and target board family [21]. *Scenario:* an instruction succeeds in English but degrades in Bangla, or works for an Uno but not an ESP32 variant. *Why misattributed:* with no view of a language- or board-dependent code path, variance is attributed to the effector’s condition.

WM2 (attributional, all layers) — *one permanent failure.* *Cause:* a failed command can originate in the microphone, the motors, a thermal cutoff, the generated artifact, or the deploy step—each needing a different fix, none permanent. *Scenario:* the robot does not move; the injected fault is in synthesis, but the microphone indicator was lit and the motors are sound. *Why misattributed:* lacking layer-specific feedback, the user collapses distinct faults into “it doesn’t work” and may discard functioning hardware. Appropriate reliance requires that users can tell capable from incapable states [28], and feedback about failure is what makes trust track reality [10].

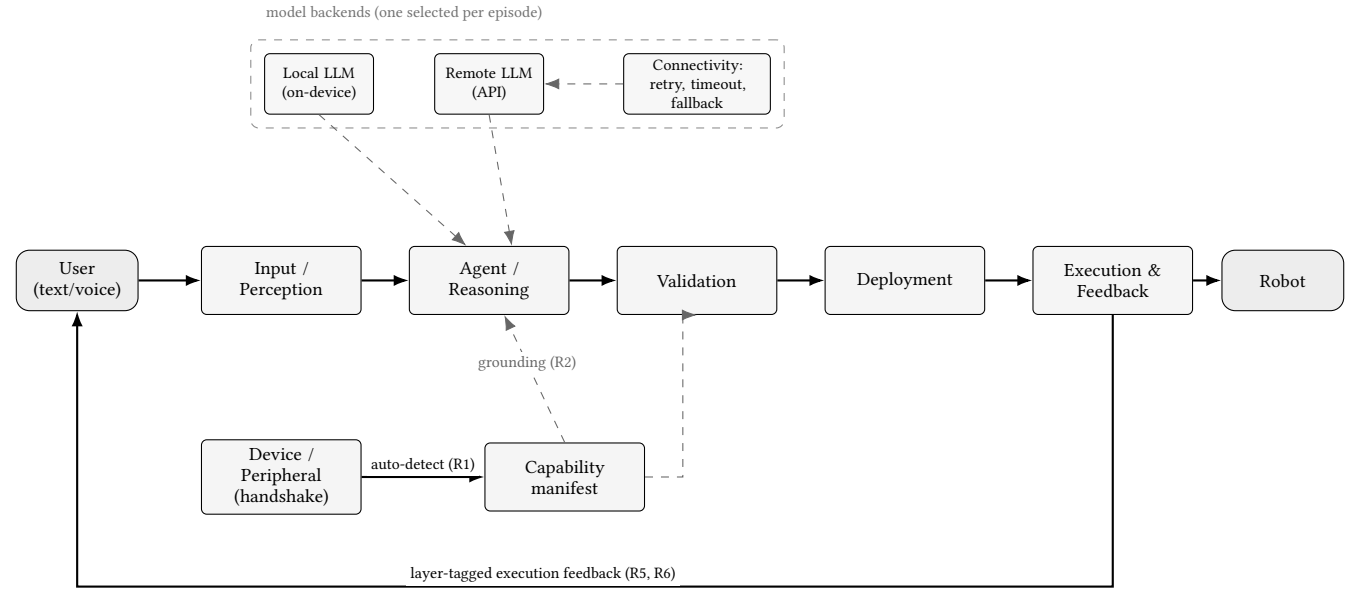


Figure 2: System architecture. Solid arrows are the runtime command path; dashed arrows are model selection, connectivity handling, and capability grounding. Module names match §4 and the requirement IDs in Table 1.

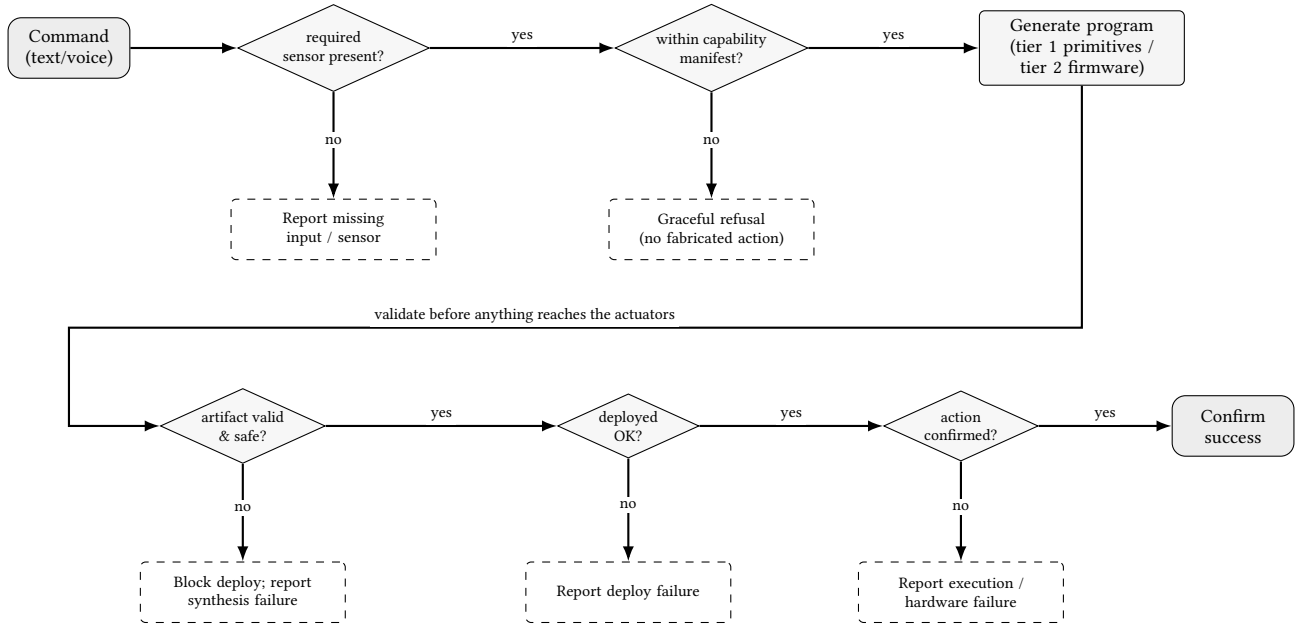


Figure 3: Operational flow for one command, read left to right on the top row and continuing on the bottom row. Each decision routes failure to a distinct, labelled report so the interface can localize the cause (R6). This is the design counterpart to WM2 in §5; the layer tag on each exit is what the interface shows the user and what §6 scores fault-localization against.

WM4 (attributional, context) — “it forgot / it ignored me.” Cause: a follow-up can fail through context truncation, a token limit, an API error, or latency. Scenario: “do that again, but slower” fails because the earlier action has fallen out of context. Why misattributed: the user infers disobedience or unreliable memory rather than a context

boundary; long-context models use context unevenly [31] and the state is invisible unless surfaced (R8).

WM3 (boundary) — “it can do anything I say.” Cause: real capability is bounded by hardware; without an exposed manifest the user infers capability from linguistic expressibility. Scenario: a command

requires an absent sensor and the system partially executes rather than refusing. *Why misattributed*: partial or fabricated behaviour reads as “it tried and nearly did it.” SayCan frames exactly this [1]; guideline G1 targets it [3].

WM5 (temporal, within session) — “it always works.” *Cause*: reliability is below the level a streak of successes implies. *Scenario*: after several flawless commands the user delegates a higher-stakes action unchecked. *Why misattributed*: past success is read as a guarantee—the classic over-trust failure of automation [28, 37], amplified here because users under-detect faults in generated artifacts [38, 44].

WM7 (temporal, across sessions) — “it used to work.” *Cause*: a silent model or firmware update changes behaviour between sessions. *Scenario*: a command that worked last week behaves differently after an unannounced swap. *Why misattributed*: holding a model of a static artifact, the user reads change as malfunction or as their own error; expectation violation from unannounced algorithmic change drives confusion and resistance [11], and notifying about change is a guideline [3] captured by R10.

5.4 Applying the frame

Because no data have been collected, we do not illustrate the taxonomy with personas or invented utterances; synthetic examples would read as findings regardless of hedging, and they cannot demonstrate that a coding frame survives contact with real talk. What we can specify in advance are the boundary rules that make coding decidable, since these are the places where two coders will diverge:

- **WM1 vs. WM6.** If the participant’s account attributes a *degraded property to the effector* (“the robot got worse”), code WM1; if it attributes *understanding or volition* (“the robot didn’t want to”), code WM6. Statements doing both are double-coded, and the ratio of double-coding is itself reported.
- **WM2 vs. WM3.** WM2 is a wrong *cause* for a failure that was in principle recoverable; WM3 is a wrong *boundary* belief about what was ever possible.
- **WM4 vs. WM2.** WM4 requires reference to a prior turn; a context failure described without that reference is WM2.
- **WM5 vs. WM7.** WM5 is within-session over-generalization from successes; WM7 requires a cross-session comparison.

6 Study 1: Testing the Taxonomy

6.1 Design rationale

Mental models are inferred, not observed [36, 43], so we plan mixed methods: think-aloud surfaces in-the-moment reasoning, a post-scenario interview probes stable beliefs, and a structured instrument yields a comparable score. We split the work into two phases, because a single design cannot both afford think-aloud depth and support the group comparisons the research questions need.

Phase A (depth), $n \approx 36$, 12 per stratum, in-lab, 60 minutes, think-aloud plus interview plus instrument. This phase carries RQ1 and RQ2 and is the source of qualitative evidence. **Phase B (breadth)**, $n \approx 120$, remote, 20 minutes, instrument-only over

recorded episodes of the same scenarios. This phase estimates category incidence with usable precision and tests the RQ2 mediation without requiring 100 think-aloud sessions—which, at 45–60 minutes plus transcription and double-coding, is not a feasible commitment for the analysis we describe. Strata are (G1) no computing or robotics background; (G2) hobbyist/intermediate; (G3) CS or robotics expert. Stratification is central rather than incidental: the hypothesis is that breakdowns differ systematically with prior exposure, so background is an independent variable.

6.2 Scenario design, and avoiding a confirmatory study

A protocol with exactly one scripted fault per taxonomy category can only recover the categories it was built from. We therefore use three scenario blocks. **Block 1 (scripted, 5 episodes)** injects one fault per layer of Figure 3—a disabled microphone, a capability-exceeding request, an injected synthesis error, a deploy failure, a stalled actuator—in counterbalanced order. **Block 2 (cross-session, 1 episode)** returns Phase A participants after an unannounced model swap, targeting WM7 without telling participants that anything changed. **Block 3 (open, 10–12 minutes)** is free use with no injected faults, in which whatever fails, fails; this block is the one that can surface categories the taxonomy lacks, and its transcripts are coded inductively first, with the deductive frame applied only afterwards by a coder blind to Block 1 and 2 assignments.

Incidence estimated from Block 1 is *conditional on the scenario* and is not a population base rate. We will report it as such, and use Block 3 for any statement about spontaneous prevalence.

6.3 The mental-model-accuracy instrument

The instrument is specified here because “a mental-model-accuracy score” is otherwise unfalsifiable. A correct model is defined operationally, against the reference decomposition in Figure 2, as a functional model adequate for action—not the engineer’s model. It has four components:

- (1) **Structural placement (RQ2 predictor).** Given the six layer names in a scrambled order and three short behaviour descriptions, place where interpretation, program-writing, and execution happen. Scored 0–3.
- (2) **Fault localization (primary outcome).** For each of five recorded failure episodes, select the responsible layer from six options plus “can’t tell.” Scored as accuracy and, separately, as a confusion matrix over layers, which is what tells us *how* attribution fails, not just how often.
- (3) **Boundary prediction.** Twelve commands, half within and half outside the manifest; judge executable or not. Scored with d' so that response bias is separated from discrimination—a participant who answers “yes” to everything is exactly the WM3 case and must not score as accurate.
- (4) **Calibration.** Confidence on each prediction item, scored as a Brier score; over-confidence is the WM5 signal and is the appropriate-reliance outcome [28].

Per-category presence/absence judgements come from the qualitative coding, not from the instrument, so that the two are not circular. The instrument requires piloting: item wording, the six-way layer

Table 2: Taxonomy of mental-model breakdowns, organized by belief class and by the pipeline layer where the true cause lives. Each category links to a requirement and a mitigation (§7). Categories are a coding frame, not a partition: an episode may instantiate several.

ID	Class	True locus	User’s belief	True cause	Req.	Mit.
WM6	Structural	Agent vs. effector	“The robot understood and chose”	Reasoning is in the hidden agent; the robot executes	R7	M4
WM1	Attributional	Agent (synthesis)	“It’s broken / it got dumber”	Language- and board-dependent competence gap	R6	M5
WM2	Attributional	Any layer	“It doesn’t work” (one permanent fault)	Distinct faults: mic, motor, thermal, synthesis, deploy	R6	M1
WM4	Attributional	Agent (context)	“It forgot / it ignored me”	Context truncation, token limit, API error, latency	R8	M3
WM3	Boundary	Device / manifest	“It can do anything I say”	Capability bounded by hardware; manifest not exposed	R1, R2	M2
WM5	Temporal (session)	Whole pipeline	“It always works”	Reliability below the level a success streak implies	R9	M6
WM7	Temporal (cross-session)	Governance	“It used to work”	Silent model or firmware update changed behaviour	R10	M6

choice, and the manifest-boundary items all need a pretest of 8–10 participants, and its internal structure should be checked before Phase B rather than reported post hoc. Bangla and English versions will be produced by translation and independent back-translation.

6.4 Analysis

Transcripts are analyzed with a codebook (template) thematic analysis rather than reflexive thematic analysis. This is a deliberate correction: reflexive TA treats coding as interpretive and explicitly does not target coder agreement [6], so computing an agreement coefficient over a reflexive analysis is incoherent, and HCI guidance is clear that agreement statistics belong to codebook-style analyses with a positivist warrant [33]. Because our deductive layer is a predefined frame we intend to test, codebook TA is the consistent choice, and we report agreement accordingly.

Two researchers independently code a shared 25% subset. Because segments may carry multiple labels, agreement is reported with Krippendorff’s α for multi-valued data [25] rather than Cohen’s κ , which assumes single-label nominal judgements [9]. Disagreements are resolved by discussion, the codebook is revised, and the revision history is reported rather than a bare coefficient [33]. Inductive codes from Block 3 are developed before the deductive frame is applied to that block.

Quantitatively, the **primary outcome** is fault-localization accuracy (component 2) compared across strata; everything else is secondary and labelled as such. We preregister hypotheses, outcomes, exclusions, and the analysis before data collection [8]. Reporting is by effect sizes with interval estimates rather than dichotomous significance tests [13]; where a test is used across the seven categories, we apply Holm correction and report denominators. With $n \approx 40$ per stratum in Phase B, a between-stratum difference in a per-category incidence proportion is estimable to roughly ± 0.15 at 95% confidence, which we state up front so that small differences are not over-read. RQ2 is tested as a mediation: structural placement score $\rightarrow$ fault-localization accuracy, with technical background as a covariate.

6.5 Ethics and safety

The study requires ethics-board approval before running. Fault injection means participants are told that some failures may be experimenter-controlled but not which ones—a mild deception that requires justification, a debriefing at the end of each session disclosing every injected fault, and an option to withdraw data after debriefing. Participation is voluntary and compensated at or above the local hourly norm; participants may skip items or withdraw without penalty; sessions are pseudonymized; only data required for the analysis are retained, under access control. R5 governs system-side logs, not participant identifiers.

Because the effector is physical, sessions run on a low-torque platform inside a bounded arena, with a hardware e-stop within the participant’s reach, and the tier-2 firmware path disabled for all participant sessions. Participants choose Bangla or English for both the session and the instrument, and the consent form is available in both.

7 Study 2: Evaluating Mitigations

7.1 Mitigations

Each mitigation targets a category, traces to a requirement, and instantiates a human–AI interaction guideline [3]:

M1 Layer-specific status distinguishing input, capability, synthesis, deploy, and execution failures (WM2; R6).

M2 Surfaced capability manifest so the agent refuses gracefully instead of fabricating (WM3; R1–R2).

M3 Context/session indicator showing when earlier commands have expired (WM4; R8).

M4 Pipeline legibility cues naming the active stage (WM6; R7), the mitigation that most directly targets layer collapse [15].

M5 Language and board indicator plus code-switch-aware handling, flagging when the request is on a weaker generation path (WM1; motivated by [21]).

M6 Calibration and change cues reliability framing after early successes and a behavioural-diff change notice (WM5, WM7; R9–R10; cf. [24]).

7.2 Design

Evaluating the six mitigations as a single “transparent” bundle against an opaque baseline cannot attribute an effect to any one of them, which is what the questions below require. We therefore use a 2×2 between-subjects core—structural transparency (M1, M4) $\times$ boundary/state transparency (M2, M3)—with M5 and M6 as within-subjects manipulations triggered by scenario. This isolates the layer-collapse prediction: if structural transparency mediates the effect, the M1/M4 factor should improve fault-localization accuracy across categories, including those it does not directly address, whereas the M2/M3 factor should improve only its own categories.

Evaluation questions: (1) Does each factor lower the incidence of its targeted categories relative to baseline? (2) Does layer-specific feedback improve fault-localization accuracy (WM2)? (3) Do legibility and manifest cues reduce over-attribution and boundary errors (WM3, WM6) without inflating task time or workload, measured with NASA-TLX [18]? (4) Does the structural factor produce cross-category transfer that the non-structural factor does not—the discriminating test of the layer-collapse account? Explanation dose is itself a variable, since more explanation is not uniformly better [27], and we hold total text volume roughly constant across conditions so that the structural manipulation is not confounded with verbosity.

7.3 Success criteria

Success criteria derive from the taxonomy: for each targeted category, the mitigated configuration should show lower incidence and better fault-localization, with calibration (Brier) improving rather than confidence merely rising. Any safety-relevant breakdown pauses iteration until reviewed. Results feed back into the requirements rather than certifying the system as finished.

8 Limitations and Threats to Validity

The central limitation is status: this is a framework and a protocol, so the taxonomy is grounded in literature and in the system’s failure surface, not in observation. Its categories may prove to be one construct or ten.

The most serious threat to the eventual study is separating a genuine mental-model breakdown from a correct reaction to a real system failure. The design mitigates this with scripted, fully logged fault injection (R5) so that every observed failure has a known cause, but Block 3 has no such guarantee and its coding must mark cause-unknown episodes explicitly rather than absorbing them.

Further limits: the reliability of the generator itself, which if poor will dominate sessions and must be measured and reported per condition; the specificity of the ESP32/Arduino platform and of a two-tier design that many deployed systems do not share; and the cultural and linguistic specificity of a Bangladesh-based sample—though the Bangla/English contrast is an asset for WM1 rather than only a threat to generalization. Deception, even mild, may alter how participants report causes, and the debriefing protocol should

be evaluated in the pilot. Finally, the authors’ engineering background may bias coding toward mechanism-centric categories; the inductive-first ordering of Block 3 and a second coder are intended as checks, and we report coder backgrounds.

9 Conclusion

When people command a physical robot through an LLM agent that silently writes and deploys its program, the middle layer is absent from what they can perceive and, we argue, absent from the model they build. Layer collapse explains why the resulting errors are not a scattered list: they are the downstream consequences of a single structural omission, and they are therefore addressable by restoring structure rather than by adding explanation. The paper contributes that account, a taxonomy organized by it, a system design in which each category has a corresponding interface obligation, and two protocols—one to test the taxonomy and one to test whether the structural intervention transfers across categories as the account predicts. The immediate next step is the instrumented testbed of \$4 and a think-aloud pilot with 8–10 non-experts, which will refine the coding frame and the instrument before either study is preregistered. A designer building one of these systems can use the taxonomy and requirements here to make the hidden layer legible before users learn it wrong.

References

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishk Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*.
- [2] Gopika Ajaykumar, Maureen Steele, and Chien-Ming Huang. 2021. A Survey on End-User Robot Programming. *Comput. Surveys* 54, 8 (2021), 1–36. doi:10.1145/3466819
- [3] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fournay, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–13. doi:10.1145/3290605.3300233
- [4] Gagan Bansal, Besmira Nushi, Ece Kamar, Walter S. Lasecki, Daniel S. Weld, and Eric Horvitz. 2019. Beyond Accuracy: The Role of Mental Models in Human-AI Team Performance. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (HCOMP)*, Vol. 7. 2–11.
- [5] bkrajendra. 2024. arduino-llm-agent: Generate, Compile, and Upload Arduino/ESP8266/ESP32 Firmware from Natural Language using Local LLMs. GitHub repository. <https://github.com/bkrajendra/arduino-llm-agent>.
- [6] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp0630a
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [8] Andy Cockburn, Carl Gutwin, and Alan Dix. 2018. HARK No More: On the Preregistration of CHI Experiments. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–12. doi:10.1145/3173574.3173715
- [9] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. doi:10.1177/001316446002000104
- [10] Munjal Desai, Poornima Kanarasu, Mikhail Medvedev, Aaron Steinfeld, and Holly Yanco. 2013. Impact of Robot Failures and Feedback on Real-Time Trust. In *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. 251–258. doi:10.1109/HRI.2013.6483596

- [11] Michael A. DeVito, Darren Gergle, and Jeremy Birnholtz. 2017. "Algorithms Ruin Everything": #RIPTwitter, Folk Theories, and Resistance to Algorithmic Change in Social Media. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 3163–3174. doi:10.1145/3025453.3025659
- [12] Anca D. Dragan, Kenton C. T. Lee, and Siddhartha S. Srinivasa. 2013. Legibility and Predictability of Robot Motion. In *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. 301–308. doi:10.1109/HRI.2013.6483603
- [13] Pierre Dragicevic. 2016. Fair Statistical Communication in HCI. In *Modern Statistical Methods for HCI*. Springer, 291–330. doi:10.1007/978-3-319-26633-6_13
- [14] Motahhare Eslami, Karrie Karahalios, Christian Sandvig, Kristen Vaccaro, Aimee Rickman, Kevin Hamilton, and Alex Kirlik. 2016. First I "Like" It, Then I Hide It: Folk Theories of Social Feeds. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 2371–2382. doi:10.1145/2858036.2858494
- [15] Motahhare Eslami, Aimee Rickman, Kristen Vaccaro, Amirhossein Aleyasen, Andy Vuong, Karrie Karahalios, Kevin Hamilton, and Christian Sandvig. 2015. "I Always Assumed That I Wasn't Really That Close to [Her]": Reasoning about Invisible Algorithms in News Feeds. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. 153–162. doi:10.1145/2702123.2702556
- [16] Katy Ilonka Gero, Zahra Ashktorab, Casey Dugan, Qian Pan, James Johnson, Werner Geyer, Maria Ruiz, Sarah Miller, David R. Millen, Murray Campbell, Sadhana Kumaravel, and Wei Zhang. 2020. Mental Models of AI Agents in a Cooperative Game Setting. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–12. doi:10.1145/3313831.3376316
- [17] Peter A. Hancock, Deborah R. Billings, Kristin E. Schaefer, Jessie Y. C. Chen, Ewart J. de Visser, and Raja Parasuraman. 2011. A Meta-Analysis of Factors Affecting Trust in Human-Robot Interaction. *Human Factors* 53, 5 (2011), 517–527. doi:10.1177/0018720811417254
- [18] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*. Advances in Psychology, Vol. 52. North-Holland, 139–183. doi:10.1016/S0166-4115(08)62386-9
- [19] Shanee Honig and Tal Oron-Gilad. 2018. Understanding and Resolving Failures in Human-Robot Interaction: Literature Review and Model Development. *Frontiers in Psychology* 9 (2018), 861. doi:10.3389/fpsyg.2018.00861
- [20] Philip N. Johnson-Laird. 1983. *Mental Models: Towards a Cognitive Science of Language, Inference, and Consciousness*. Harvard University Press, Cambridge, MA.
- [21] Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. 2020. The State and Fate of Linguistic Diversity and Inclusion in the NLP World. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 6282–6293. doi:10.18653/v1/2020.acl-main.560
- [22] David E. Kieras and Susan Bovair. 1984. The Role of a Mental Model in Learning to Operate a Device. *Cognitive Science* 8, 3 (1984), 255–273. doi:10.1207/s15516709cog0803_3
- [23] Taemie Kim and Pamela Hinds. 2006. Who Should I Blame? Effects of Autonomy and Transparency on Attributions in Human-Robot Interaction. In *Proceedings of the 15th IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN)*. 80–85. doi:10.1109/ROMAN.2006.314398
- [24] Rafal Kocielnik, Saleema Amershi, and Paul N. Bennett. 2019. Will You Accept an Imperfect AI? Exploring Designs for Adjusting End-User Expectations of AI Systems. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–14. doi:10.1145/3290605.3300641
- [25] Klaus Krippendorff. 2018. *Content Analysis: An Introduction to Its Methodology* (4th ed.). SAGE Publications, Thousand Oaks, CA.
- [26] Todd Kulesza, Simone Stumpf, Margaret Burnett, and Irwin Kwan. 2012. Tell Me More? The Effects of Mental Model Soundness on Personalizing an Intelligent Agent. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1–10. doi:10.1145/2207676.2207678
- [27] Todd Kulesza, Simone Stumpf, Margaret Burnett, Sherry Yang, Irwin Kwan, and Weng-Keen Wong. 2013. Too Much, Too Little, or Just Right? Ways Explanations Impact End Users' Mental Models. In *2013 IEEE Symposium on Visual Languages and Human Centric Computing (VL/HCC)*. 3–10. doi:10.1109/VLHCC.2013.6645235
- [28] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80. doi:10.1518/hfes.46.1.50.30392
- [29] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as Policies: Language Model Programs for Embodied Control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 9493–9500. doi:10.1109/ICRA48891.2023.10160591
- [30] Q. Vera Liao and Jennifer Wortman Vaughan. 2024. AI Transparency in the Age of Large Language Models: A Human-Centered Research Roadmap. *Harvard Data Science Review Special Issue* 5 (2024). doi:10.1162/99608f92.8036d03b
- [31] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638
- [32] Ewa Luger and Abigail Sellen. 2016. "Like Having a Really Bad PA": The Gulf between User Expectation and Experience of Conversational Agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 5286–5297. doi:10.1145/2858036.2858288
- [33] Nora McDonald, Sarita Schoenebeck, and Andrea Forte. 2019. Reliability and Inter-rater Reliability in Qualitative Research: Norms and Guidelines for CSCW and HCI Practice. *Proceedings of the ACM on Human-Computer Interaction* 3, CSCW (2019), 1–23. doi:10.1145/3359174
- [34] Clifford Nass and Youngme Moon. 2000. Machines and Mindlessness: Social Responses to Computers. *Journal of Social Issues* 56, 1 (2000), 81–103. doi:10.1111/0022-4537.00153
- [35] Don Norman. 2013. *The Design of Everyday Things: Revised and Expanded Edition*. MIT Press, Cambridge, MA.
- [36] Donald A. Norman. 1983. Some Observations on Mental Models. In *Mental Models*, Dedre Gentner and Albert L. Stevens (Eds.). Lawrence Erlbaum Associates, Hillsdale, NJ, 7–14.
- [37] Raja Parasuraman and Victor Riley. 1997. Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors* 39, 2 (1997), 230–253. doi:10.1518/001872097778543886
- [38] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157
- [39] Jennifer Preece, Yvonne Rogers, and Helen Sharp. 2023. *Interaction Design: Beyond Human-Computer Interaction* (6th ed.). John Wiley & Sons, Indianapolis, IN.
- [40] Emilee Rader and Rebecca Gray. 2015. Understanding User Beliefs About Algorithmic Curation in the Facebook News Feed. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. 173–182. doi:10.1145/2702123.2702174
- [41] Maha Salem, Gabriella Lakatos, Farshid Amirabdollahian, and Kerstin Dautenhahn. 2015. Would You Trust a (Faulty) Robot? Effects of Error, Task Type and Personality on Human-Robot Cooperation and Trust. In *Proceedings of the 10th Annual ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. 141–148. doi:10.1145/2696454.2696497
- [42] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2023. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 11523–11530. doi:10.1109/ICRA48891.2023.10161317
- [43] Aaqib Tabrez, Matthew B. Luebbbers, and Bradley Hayes. 2020. A Survey of Mental Modeling Techniques in Human–Robot Teaming. *Current Robotics Reports* 1, 4 (2020), 259–267. doi:10.1007/s43154-020-00019-0
- [44] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–7. doi:10.1145/3491101.3519665
- [45] Sai Vemprala, Rogerio Bonatti, Arthur Buckner, and Ashish Kapoor. 2024. ChatGPT for Robotics: Design Principles and Model Abilities. *IEEE Access* 12 (2024), 55682–55696. doi:10.1109/ACCESS.2024.3387941
- [46] Jacob O. Wobbrock and Julie A. Kientz. 2016. Research Contributions in Human-Computer Interaction. *Interactions* 23, 3 (2016), 38–44. doi:10.1145/2907069